



Отчет предоставлен сервисом
«Антиплагиат» - <http://vsuet.antiplagiat.ru>



Отчет о проверке

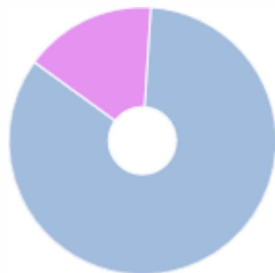
Автор: Лемешкин Александр Викторович

Имя документа: ВКР_Звхврин_BC_StoryDB_оформлено

Проверяющий: Лемешкин Александр Викторович

Организация: Федеральное государственное бюджетное образовательное учреждение высшего образования «Воронежский государственный университет инженерных технологий»

РЕЗУЛЬТАТЫ ПРОВЕРКИ



Машинно сгенерированный текст:
21,7%



Текст, написанный человеком:
78,3%



⚠ Ряд характеристик этого текста указывают на то, что текст, возможно, был сгенерирован при помощи искусственного интеллекта. Однако, окончательное решение о качестве текста остается за проверяющим экспертом

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 546

Тип документа: Не указано

Дата проверки: 23.06.2026 14:17:38

Дата корректировки: Нет

Количество страниц: 42

Символов в тексте: 56889

Слов в тексте: 6644

Число предложений: 566

ИСХОДНЫЙ ТЕКСТ

Реферат

Выпускная квалификационная работа посвящена разработке веб-приложения StoryDB, предназначенного для авторов художественных произведений и команд, работающих с крупными сюжетными проектами. В работе рассматриваются проблемы хранения разрозненных сведений о персонажах, предметах, местах, организациях, событиях, отношениях и изменениях сюжетного мира. Предложено решение в виде единой веб-системы, объединяющей базу объектов, настраиваемые каталоги, граф связей, структуры, временную линию, загрузку изображений, публикацию снимков проекта и экспорт досье в формат DOCX.

Практическим результатом работы является полнофункциональное веб-приложение, реализованное на основе клиент-серверной архитектуры. Серверная часть выполнена на ASP.NET Core с применением Entity Framework Core и PostgreSQL, клиентская часть разработана на React и TypeScript. Для развертывания используется Docker Compose. В системе предусмотрены аутентификация пользователей, разграничение доступа к проектам, аудит действий, ограничение ресурсоемких запросов, обработка медиафайлов, фоновые задания экспорта, эксплуатационные эндпоинты состояния и метрик.

Ключевые слова: веб-приложение, художественное произведение, StoryDB, база знаний, сюжетный проект, персонажи, граф связей, таймлайн, каталогизация, ASP.NET Core, React, PostgreSQL.

Содержание

Введение

1 Анализ предметной области и постановка задачи

1.1 Особенности информационной поддержки авторов художественных произведений

1.2 Анализ существующих подходов и программных средств

1.3 Требования к разрабатываемому веб-приложению

1.4 Выбор технологического стека

2 Проектирование веб-приложения StoryDB

2.1 Общая архитектура системы

2.2 Информационная модель проекта художественного произведения

2.3 Проектирование серверной части и API

2.4 Проектирование клиентской части

2.5 Проектирование механизмов безопасности и эксплуатации

3 Реализация веб-приложения

3.1 Реализация серверной части

3.2 Реализация пользовательского интерфейса

3.3 Реализация графа связей, структур и временной линии

3.4 Реализация медиафайлов, снимков проекта и экспорта

3.5 Развертывание приложения

4 Тестирование и оценка результатов

4.1 Методика тестирования

4.2 Проверка функциональных сценариев

4.3 Оценка соответствия требованиям

4.4 Перспективы развития

Заключение

Список использованных источников

Приложение А. Перечень иллюстраций для последующей вставки

Введение

Современная работа над художественным произведением все чаще

выходит за пределы обычного текстового редактора. Автору необходимо

помнить не только основной текст, но и множество связанных сведений:

биографии персонажей, названия организаций, географию мира, свойства

предметов, причинно-следственные связи, хронологию событий, изменения

статусов и отношения между участниками сюжета. Чем крупнее

произведение, тем сложнее поддерживать внутреннюю непротиворечивость материала.

В небольшом рассказе сведения можно удерживать в памяти или в

одном файле заметок. Однако при разработке романа, цикла произведений,

сценария, настольной игры или игрового мира количество сущностей быстро

растет. Одна и та же информация начинает повторяться в таблицах, текстовых

документах, заметках, картинках и переписке. В результате автор

сталкивается с дублированием данных, потерей актуальных версий,

сложностью поиска и риском противоречий.

Особенно заметной становится проблема связей. Персонаж может

состоять в организации, владеть предметом, находиться в определенном

месте, иметь родственные, конфликтные или союзнические отношения с

другими персонажами. Эти отношения могут меняться во времени. Если

связи описаны только словами в разных документах, автору сложно быстро

оценить общую картину сюжетного мира и проверить, не нарушена ли логика повествования.

Актуальность темы обусловлена потребностью в специализированных

цифровых инструментах, которые помогают авторам художественных

произведений системно хранить и анализировать материалы проекта.

Универсальные заметочки и таблицы позволяют фиксировать сведения, но

не всегда поддерживают предметную модель художественного мира: типы

объектов, настраиваемые поля, каталоги, иерархии, граф отношений,

временную линию и экспорт структурированных досье.

Объектом исследования является процесс информационного

сопровождения разработки художественного произведения. Предметом

исследования является веб-приложение, обеспечивающее хранение,

структурирование, визуализацию и экспорт данных сюжетного проекта.

Цель выпускной квалификационной работы - разработать веб-

приложение для авторов художественных произведений, позволяющее вести

единую базу знаний проекта, управлять объектами сюжетного мира,

анализировать связи между ними и фиксировать хронологию событий.

Для достижения поставленной цели необходимо решить следующие

задачи: проанализировать предметную область и существующие подходы к

организации авторских материалов; сформировать функциональные и

нефункциональные требования к приложению; выбрать технологический

стек; спроектировать архитектуру, модель данных и пользовательские

сценарии; реализовать серверную и клиентскую части приложения;

предусмотреть механизмы безопасности, хранения медиафайлов, экспорта и

эксплуатации; провести проверку основных функций.

В работе использованы методы системного анализа, объектного

моделирования, проектирования реляционных баз данных, клиент-серверной

разработки, функционального тестирования и анализа пользовательских сценариев.

Практическая значимость работы заключается в создании инструмента, который может использоваться авторами, редакторами, сценаристами и разработчиками сюжетных проектов для уменьшения количества противоречий в материалах, ускорения поиска сведений, подготовки досье и визуального анализа связей.

Выпускная квалификационная работа состоит из введения, четырех глав, заключения, списка использованных источников и приложения. В первой главе рассматривается предметная область и формулируются требования. Во второй главе описывается проектирование системы. В третьей главе приведена реализация основных модулей. В четвертой главе рассматриваются тестирование, оценка результата и направления дальнейшего развития.

1 Анализ предметной области и постановка задачи

1.1 Особенности информационной поддержки авторов художественных произведений

Художественный проект представляет собой не только итоговый текст, но и совокупность вспомогательных материалов. Автор формирует описания персонажей, локаций, предметов, организаций, правил мира, исторических событий, временных периодов, конфликтов и сюжетных линий. Эти сведения используются при написании сцен, редактировании, подготовке продолжений, согласовании деталей с редактором или командой.

На раннем этапе работы объем таких материалов обычно невелик.

Автор может вести их в виде отдельных заметок, таблиц или документов.

Однако по мере развития проекта структура данных усложняется.

Появляются повторяющиеся типы объектов, общие характеристики, классификации, пересекающиеся сюжетные линии и связи между несколькими сущностями. При отсутствии единой модели автор вынужден вручную синхронизировать сведения между разными файлами.

К основным проблемам предметной области можно отнести фрагментацию информации, отсутствие единой точки доступа, слабую поддержку отношений между объектами, невозможность быстро увидеть изменения во времени, ограниченные средства поиска по смысловым группам и сложность подготовки структурированного отчета по выбранным персонажам или объектам.

Для автора важна не только фиксация фактов, но и возможность анализировать их в разных разрезах. Например, требуется увидеть всех персонажей, связанных с организацией, определить, какие предметы находятся у героя в конкретный момент времени, проверить, какие события привели к конфликту, или подготовить досье для редактора. Такие операции неудобно выполнять в обычном текстовом редакторе.

Следовательно, предметная область требует инструмента, который совмещает свойства базы знаний, справочника, визуальной карты связей и хронологической модели. При этом приложение должно оставаться достаточно простым для автора, который не обязан обладать навыками администрирования баз данных.

1.2 Анализ существующих подходов и программных средств

Для ведения материалов художественного проекта авторы используют разные классы инструментов. К первому классу относятся текстовые редакторы и облачные документы. Они удобны для написания основного

текста, но плохо поддерживают структурированные связи между сущностями.

При большом объеме заметок поиск становится зависимым от дисциплины автора и качества ручной разметки.

Ко второму классу относятся электронные таблицы. Они позволяют хранить однотипные записи, фильтровать и сортировать данные, но требуют самостоятельного проектирования структуры. Таблицы плохо подходят для многоуровневых отношений, графов, временных изменений и вложенных справочников. Кроме того, они не создают полноценного пользовательского интерфейса для работы с художественным миром.

Третий класс составляют универсальные системы заметок и базы знаний. Они дают гибкость и позволяют связывать страницы между собой, однако часто не имеют предметных сущностей, специализированных карточек, ограничений целостности и визуальных режимов, ориентированных именно на художественный проект.

Четвертый класс - специализированные инструменты для писателей и worldbuilding-системы. Они ближе к предметной области, но не всегда позволяют адаптировать структуру под конкретный проект, развернуть систему локально, контролировать хранение данных, расширить модель каталогами и собственными типами связей.

Таблица 1.1 - Сравнение подходов к ведению материалов художественного проекта

Подход Преимущества Ограничения для задачи

Текстовые Простота, привычный

формат, удобство

Нет строгой модели объектов,

сложно анализировать связи и

Проведенный анализ показывает, что целесообразно разрабатывать

специализированное веб-приложение, которое сочетает гибкость

настраиваемой базы знаний и предметные инструменты автора: карточки

объектов, каталоги, структуры, граф отношений, временную линию, загрузку

изображений и экспорт досье.

Важным критерием при выборе подхода является возможность

долговременного сопровождения проекта. Художественное произведение

может развиваться годами: появляются новые редакции, дополнительные

сюжетные ветви, второстепенные персонажи, приложения, справочные

материалы и продолжения. Если инструмент не поддерживает расширение

структуры данных, автор вынужден либо менять способ ведения материалов,

либо сохранять часть сведений вне основной системы.

Другим критерием является переносимость данных. Внешние облачные

сервисы удобны для быстрого старта, но создают зависимость от конкретной

платформы и ее тарифной политики. Локально разворачиваемая веб-система

позволяет контролировать хранение данных, резервное копирование и состав

доступных функций. Для учебного и исследовательского проекта это также

упрощает демонстрацию архитектуры и проверку программной реализации.

Отдельно следует учитывать различие между текстом произведения и

базой знаний произведения. Текстовый редактор помогает создавать главы,

сцены и диалоги. База знаний отвечает на другие вопросы: кто связан с кем,

какие свойства имеет объект, где находится персонаж, какие события

документы написания сцен изменения

Электронные

таблицы

- Фильтрация, сортировка,
- табличная структура
- Неудобны для графов отношений,
- вложенных структур и медиа
- Универсальные
- заметочки
- Гибкость, ссылки между
- страницами, быстрые
- заметки
- Мало предметных ограничений, нет
- единого API и специализированного
- экспорта
- Специализиров
- анные сервисы
- Близость к задачам
- авторов, готовые
- шаблоны
- Ограниченная адаптация,
- зависимость от внешнего сервиса, не
- всегда есть локальное
- развертывание

изменили его статус, какие элементы относятся к выбранной структуре.

Поэтому разрабатываемое приложение не заменяет редактор текста, а дополняет его как инструмент структурирования и анализа.

1.3 Требования к разрабатываемому веб-приложению

На основе анализа предметной области сформулированы функциональные и нефункциональные требования. Функциональные требования описывают действия пользователя и основные возможности системы. Нефункциональные требования определяют качество работы приложения, требования к безопасности, эксплуатации и расширяемости.

Таблица 1.2 - Функциональные требования к веб-приложению

Группа	требований	Содержание требования
Проекты	Создание, редактирование и удаление проектов художественных произведений; настройка видимости и типов объектов.	
Объекты	Ведение карточек персонажей, предметов, мест, организаций и иных сущностей с описанием, статусом, изображениями и характеристиками.	
Атрибуты	Настройка пользовательских полей, групп характеристик, числовых и выборочных значений.	
Каталоги	Создание справочников с группами, полями, ссылками между записями и иерархическими режимами.	
Связи	Описание отношений между объектами и визуальное представление графа связей.	
Структуры	Построение иерархий и структур проекта, назначение объектов на узлы структуры.	
Таймлайн	Фиксация событий, длительных периодов, глав, причинно-временных связей и изменений состояния объектов.	

Медиа Загрузка изображений, подготовка файлов и привязка изображений к объектам и событиям.

Экспорт Формирование DOCX-досье по выбранным объектам с включением характеристик, каталогов, связей и структур.

Безопасность Аутентификация, сессии, проверка доступа к проектам, ограничение размеров запросов и частоты обращений.

Эксплуатация Контейнерное развертывание, эндпоинты готовности, метрики, аудит и журналирование.

К нефункциональным требованиям относятся удобство пользовательского интерфейса, модульность архитектуры, возможность расширения модели данных, устойчивость к ошибкам ввода, сохранение целостности данных, понятная структура API, возможность локального и серверного развертывания, а также наличие тестов для критичных сценариев.

Отдельным требованием является отсутствие жесткой привязки к одному жанру. Приложение должно подходить для разных типов художественных проектов: романов, циклов, сценариев, игровых миров и учебных сюжетных проектов. Поэтому система должна поддерживать настраиваемые атрибуты и каталоги, а не ограничиваться фиксированным набором полей.

С точки зрения пользователя приложение должно поддерживать несколько типовых ролей работы. Автор создает и редактирует материалы проекта. Редактор или соавтор просматривает сведения, проверяет согласованность и может работать с экспортированными досье. Владелец проекта управляет видимостью, шаблонами, снимками и доступом. Даже если в текущей версии роли не вынесены в отдельную сложную модель, архитектура должна оставлять возможность их дальнейшего развития.

Система должна обеспечивать быстрый переход от общей картины проекта к деталям. Например, автор должен иметь возможность открыть список персонажей, перейти в карточку героя, увидеть его связи, затем открыть событие таймлайна, где этот герой участвует, и вернуться к связанным объектам. Такая навигация снижает когнитивную нагрузку и поддерживает творческий процесс.

Таблица 1.3 - Основные пользовательские сценарии веб-приложения

Пользовательский

сценарий Краткое описание

Первичное наполнение

Автор создает проект, выбирает типы объектов и заносит базовые сведения о персонажах, местах и организациях.

Уточнение мира Пользователь добавляет собственные атрибуты, каталоги и справочники для особенностей жанра.

Рисунок 1.1 - Диаграмма вариантов использования веб-приложения StoryDB

1.4 Выбор технологического стека

Для реализации приложения выбран клиент-серверный подход.

Пользовательский интерфейс выполняется в браузере, а серверная часть предоставляет REST API, управляет данными, безопасностью, файлами, фоновыми заданиями и экспортом. Такой подход упрощает доступ к системе с разных устройств и позволяет разделить ответственность между клиентом и сервером.

Серверная часть реализована на ASP.NET Core. Данный фреймворк подходит для построения производительных веб-API, поддерживает

встроенную модель middleware, аутентификацию, авторизацию, dependency injection, интеграцию с OpenAPI, фоновые службы и средства настройки окружений.

Анализ связей

Автор открывает граф и проверяет отношения между персонажами, организациями, предметами и территориями.

Контроль хронологии Пользователь добавляет события, длительные периоды, главы и изменения состояния объектов.

Подготовка материалов

Система формирует DOCX-досье для выбранных объектов, которое можно передать редактору или использовать при работе над сценами.

Для доступа к данным используется Entity Framework Core с провайдером PostgreSQL. PostgreSQL выбран как надежная реляционная СУБД, поддерживающая сложные связи, индексы, транзакции и хранение JSONB-данных для снимков проекта. Такой выбор соответствует требованиям целостности и масштабируемости предметной модели.

Клиентская часть реализована на React и TypeScript. React обеспечивает компонентный подход к построению интерфейса, а TypeScript повышает надежность разработки за счет статической типизации. Для сборки используется Vite, что ускоряет локальную разработку и сборку клиентского приложения.

Для визуализации графовых структур и таймлайна в клиенте применяются специализированные библиотеки, включая React Flow и вспомогательные средства для масштабирования и компоновки. Для контейнеризации используется Docker Compose, объединяющий PostgreSQL, API и клиентское приложение.

Таблица 1.4 - Выбор технологического стека проекта

Компонент	Используемая технология	Назначение
-----------	-------------------------	------------

Сервер	ASP.NET Core, C# REST API	безопасность, бизнес-логика, фоновые задания
--------	---------------------------	--

Доступ к данным

Entity Framework Core,

Npgsql

Работа с PostgreSQL, миграции, запросы
--

База данных PostgreSQL	Хранение проектов, объектов, связей, каталогов и снимков
------------------------	--

Клиент	React, TypeScript, Vite	Пользовательский интерфейс и маршрутизация
--------	-------------------------	--

Графика

интерфейса	React Flow, D3, ELK	Визуализация графов и компоновка связей
------------	---------------------	---

Медиа	ImageSharp	браузерная
-------	------------	------------

подготовка изображений	Проверка и обработка изображений
------------------------	----------------------------------

Экспорт	OpenXML SDK	Формирование документов DOCX
---------	-------------	------------------------------

Развертывание	Docker Compose, Nginx	Запуск сервисов и публикация клиента
---------------	-----------------------	--------------------------------------

2 Проектирование веб-приложения StoryDB

2.1 Общая архитектура системы

Проектируемая система StoryDB построена как трехзвенное веб-приложение. На уровне представления находится клиентская часть, выполняемая в браузере пользователя. На серверном уровне размещается API, реализующий бизнес-логику и контроль доступа. На уровне хранения используется база данных PostgreSQL и файловое хранилище загруженных изображений.

Клиентская часть отвечает за отображение проектов, карточек объектов, каталогов, графа отношений, структур, временной линии, форм редактирования и пользовательских настроек. Серверная часть предоставляет набор API-эндпоинтов, выполняет проверку входных данных, обращается к сервисному слою, сохраняет изменения в базе данных и возвращает клиенту DTO-модели.

В архитектуре выделены контроллеры, сервисы предметной области, слой доступа к данным, сервисы безопасности, файловое хранилище, очередь аудита, сервис фонового экспорта и средства наблюдаемости. Такое разделение позволяет изолировать пользовательский интерфейс от бизнес-логики и упрощает тестирование отдельных компонентов.

Проект содержит 17 файлов контроллеров, около 111 HTTP-эндпоинтов и 45 наборов сущностей DbSet в контексте Entity Framework Core. Это показывает, что система имеет не демонстрационный, а развернутый модульный характер и охватывает несколько самостоятельных областей функциональности.

Таблица 2.1 - Уровни архитектуры веб-приложения

Уровень Основные элементы Ответственность

Клиент		
React-компоненты,		
маршрутизация, формы, графы,		
таймлайн		
Ввод, отображение и локальная		
обработка пользовательских		
данных		
API Контроллеры ASP.NET Core,		
middleware, DTO		
Прием запросов, проверка		
доступа, маршрутизация к		

Рисунок 2.1 - Архитектура веб-приложения StoryDB

Рисунок 2.2 - Контекстная IDEF0-схема процесса ведения художественного проекта	
сервисам	
Сервисный	
слой	
ProjectService, ObjectService,	
CatalogService, TimelineService и	
др.	
Бизнес-правила, валидация,	
агрегирование данных	
Хранение PostgreSQL, EF Core, файловое	
хранилище	
Долговременное хранение	

данных и медиафайлов

Эксплуатация Health endpoints, метрики, аудит,

Docker Compose

Мониторинг, запуск и

сопровождение системы

Рисунок 2.3 - Декомпозиция процесса ведения художественного проекта

2.2 Информационная модель проекта художественного произведения

Центральной сущностью системы является проект. Проект объединяет

все материалы конкретного художественного произведения или серии

произведений. С проектом связаны типы объектов, сами объекты, каталоги,

структуры, временная линия, медиафайлы, снимки и настройки доступа.

Объекты проекта представляют сущности художественного мира:

персонажей, предметы, места, организации и дополнительные типы. Каждый

объект содержит основные поля, изображение, галерею, пользовательские

атрибуты, связи с каталогами, отношения с другими объектами и участие во

временных событиях.

Для повышения гибкости введены настраиваемые атрибуты и каталоги.

Атрибуты позволяют расширять карточки объектов без изменения схемы

приложения. Каталоги используются для справочников мира: фракций, рас,

эпох, магических школ, социальных классов, технологий, терминов и других

повторно используемых сущностей. Записи каталога могут иметь поля,

группы и иерархические связи.

Модель отношений позволяет хранить связи между объектами, включая

владение, территориальную принадлежность, членство в организациях,

родственные и сюжетные связи между персонажами. Эти связи используются

как для отображения карточек, так и для построения графа отношений.

Структуры применяются для представления иерархий и схем:

организационных структур, социальных слоев, фракционных ветвей,

географических уровней или иных древовидных и графовых моделей.

Объекты и записи каталогов могут назначаться на узлы структур, что

позволяет описывать принадлежность и роль сущности в конкретной

структуре.

Таймлайн фиксирует события, периоды, главы и изменения состояния

объектов. Важной частью модели являются временные изменения: объект

может менять статус, отношения, принадлежность к каталогу или положение

в структуре. Это позволяет рассматривать художественный проект не как

статический справочник, а как динамическую систему.

При проектировании модели данных использован принцип разделения

стабильных сущностей и настраиваемых расширений. Основные таблицы

задают ядро системы: пользователи, проекты, объекты, события, каталоги и

структуры. Пользовательские атрибуты, поля каталогов и значения полей

позволяют расширять модель без добавления новой таблицы для каждого

жанра или проекта.

Для связей между сущностями используются отдельные таблицы-

связки. Такой подход облегчает поддержку отношений многие-ко-многим,

например принадлежности объекта к нескольким узлам иерархии, связи

записи каталога с другими записями или участия объекта в событии. Он

также позволяет хранить дополнительные свойства связи: роль, описание,

порядок, направление, силу или напряженность отношения.

Снимки проекта решают задачу согласованного чтения. При активной

работе данные постоянно меняются, но для публикации или просмотра важно

иметь фиксированное состояние. Поэтому текущая рабочая модель может быть преобразована в снимок, содержащий агрегированное представление проекта. Это особенно полезно для публичного просмотра и для операций, где не требуется каждый раз выполнять множество запросов к разным таблицам.

Таблица 2.2 - Группы сущностей информационной модели

Рисунок 2.4 - Укрупненная схема базы данных StoryDB

Группа сущностей Примеры таблиц / моделей Назначение

Пользователи и

доступ

AppUser, Project,

ProjectTemplatePack

Учетные записи, проекты,

шаблоны и видимость

Объекты мира StoryObject, ObjectAttribute,

ObjectRelation

Карточки сущностей,

характеристики и связи

Каталоги Catalog, CatalogEntry,

CatalogFieldDefinition

Настраиваемые справочники и

поля

Структуры

Structure, StructureNode,

StructureEdge,

StructureAssignment

Иерархии, графы и назначения

объектов

Временная линия

Timeline, TimelineEvent,

TimelineChange,

TimelineEventLink

Хронология, события и

изменения состояния

Медиа

MediaAsset,

MediaAssetVariant,

ObjectGalleryImage

Загруженные изображения и

варианты файлов

Экспорт и аудит ProjectSnapshot, AuditLog Снимки проекта, журнал

действий и подготовка отчетов

Рисунок 2.5 - Схема таблиц пользователей, проектов и публикации

Рисунок 2.6 - Схема таблиц объектов художественного мира

Рисунок 2.7 - Схема таблиц каталогов и справочников

Рисунок 2.8 - Схема таблиц структур и иерархий

Рисунок 2.9 - Схема таблиц таймлайна и изменений

Рисунок 2.10 - Схема таблиц медиафайлов и пользовательских раскладок

2.3 Проектирование серверной части и API

Серверная часть спроектирована по модульному принципу. Каждый

крупный раздел предметной области имеет собственный контроллер и сервис: проекты, объекты, атрибуты, каталоги, структуры, отношения, таймлайн, снимки проекта, экспорт, загрузки, аутентификация и аудит. Контроллеры отвечают за HTTP-интерфейс: принимают параметры маршрута, тела запросов и query-параметры, запускают проверку входных данных и преобразуют результат сервисного слоя в HTTP-ответ. Сервисный слой содержит бизнес-логику и возвращает типизированные результаты со статусами Success, NotFound, Invalid и другими состояниями.

Для обмена данными между клиентом и сервером используются DTO и request-модели. Такой подход отделяет внутренние EF-сущности от публичного API, снижает риск случайной передачи лишних полей и упрощает эволюцию модели данных.

В качестве базы данных используется PostgreSQL. Entity Framework Core выполняет отображение сущностей на таблицы, формирование запросов и применение миграций. В проекте используется миграционный подход, благодаря которому схема базы данных может последовательно развиваться вместе с приложением.

Отдельное внимание уделено операциям, потенциально влияющим на производительность: построению графов, экспорту, работе со снимками, обработке изображений и запросам к крупным спискам объектов. Для таких операций используются ограничения частоты обращений, фоновая очередь экспорта, кэширование и оптимизированные запросы.

API проектируется вокруг ресурсов проекта. Большинство маршрутов содержит идентификатор проекта, что упрощает проверку доступа и позволяет изолировать данные разных пользователей. Например, объекты, каталоги, структуры, отношения и события таймлайна запрашиваются в контексте конкретного проекта.

Для операций создания и обновления применяются request-модели. Они позволяют принимать только те поля, которые действительно должны изменяться пользователем. Для ответа применяются DTO-модели, содержащие уже подготовленные данные: имена связанных сущностей, списки ссылок, агрегированные значения и краткие представления для интерфейса.

Ошибки предметной области не пробрасываются как необработанные исключения. Сервисы возвращают результат со статусом, а контроллер преобразует его в корректный HTTP-ответ: 200, 201, 204, 400, 401, 403, 404 или 500. Такой подход делает поведение API предсказуемым для клиента.

Таблица 2.3 - Основные группы API веб-приложения

2.4 Проектирование клиентской части

Клиентская часть построена как React-приложение с маршрутизацией.

Основной пользовательский маршрут ведет в рабочую область StoryDB, где проект представлен через вкладки базы данных, связей и временной линии.

Такое разделение соответствует основным способам работы автора:

редактирование сущностей, анализ отношений и просмотр хронологии.

В рабочей области предусмотрена боковая навигация по типам

объектов, атрибутам, структурам и каталогам. Пользователь может

переключаться между персонажами, предметами, местами, организациями,

каталогами и служебными разделами. Верхняя панель содержит выбор

проекта, поиск, настройки, профиль и основные команды.

Формы редактирования разделены на логические вкладки. Например,

объект можно описывать через основные поля, атрибуты, каталоги, иерархии,

Группа API Примеры маршрутов Назначение

Проекты /api/projects Список, создание, изменение и удаление проектов

Объекты /api/projects/{projectId}/objects

CRUD карточек и работа с галереями

Каталоги /api/projects/{projectId}/catalogs

Каталоги, поля, группы и записи

Структуры /api/projects/{projectId}/structures

Структурные схемы и назначения объектов

Отношения /api/projects/{projectId}/relations

Граф связей и раскладка графа

Таймлайн /api/projects/{projectId}/timeline

События, связи событий и раскладка таймлайна

Снимки /api/projects/{projectId}/snapshots

Сборка и публикация

агрегированного состояния

Экспорт /api/projects/{projectId}/exports

Формирование и скачивание

DOCX-досье

Операционные /live, /ready, /health, /metrics Мониторинг состояния и метрик приложения

отношения и временные данные. Такой подход уменьшает перегрузку интерфейса и позволяет автору постепенно наполнять проект.

Для больших списков объектов предусмотрены механизмы виртуализации и локальной фильтрации. Это важно для крупных художественных проектов, где количество персонажей, предметов и мест может исчисляться сотнями. Интерфейс должен оставаться отзывчивым даже при большом объеме данных.

Клиент поддерживает русскую и английскую локализацию, светлую и темную тему, акцентные цветовые варианты и пользовательские настройки.

Эти возможности не являются центральной предметной логикой, но повышают удобство применения системы в реальной работе.

При проектировании интерфейса учитывалась повторяемость авторских действий. Пользователь часто выполняет небольшие операции: открыть карточку, поправить статус, добавить связь, выбрать элемент каталога, перейти к событию. Поэтому интерфейс должен минимизировать

переходы между несвязанными страницами и сохранять контекст выбранного проекта.

Для предотвращения ошибок ввода часть проверок выполняется непосредственно на клиенте. Пользователь получает сообщения о незаполненных обязательных полях, некорректных числовых диапазонах,

неверном пути изображения или неправильной настройке поля каталога до отправки запроса на сервер. Это повышает отзывчивость интерфейса, но не отменяет серверную проверку.

Большое значение имеет единообразие представления сущностей.

Карточки, детали объектов, списки, результаты поиска и элементы графа используют общие функции форматирования имен, статусов и отношений.

Это уменьшает расхождения в интерфейсе и облегчает сопровождение клиентского кода.

2.5 Проектирование механизмов безопасности и эксплуатации

Так как система хранит авторские материалы, требования безопасности являются значимыми. В приложении предусмотрена аутентификация пользователей на основе cookie-сессий. После входа пользователь получает доступ к своим проектам и к публичным проектам в зависимости от режима видимости.

Разграничение доступа реализуется на уровне фильтров и сервисов.

Для операций изменения требуется право редактирования или управления проектом. Для публичных снимков проекта допускается анонимный доступ только в режиме чтения, если владелец опубликовал соответствующий снимок.

В системе предусмотрены ограничения размера тела запроса и размеров загружаемых файлов, проверка происхождения небезопасных запросов, CORS-политика для клиентского приложения, ограничения частоты обращений для аутентификации, загрузок и ресурсоемких операций.

Для эксплуатации реализованы эндпоинты live, ready и health, а также выдача метрик в обычном и Prometheus-совместимом формате. Это позволяет проверять доступность приложения, состояние базы данных, очередей экспорта, очереди аудита, кэша и потребление памяти.

Аудит действий пользователя вынесен в отдельную очередь. Такой подход снижает влияние журналирования на время ответа основного запроса и позволяет фиксировать важные действия в проекте. Логи формируются с использованием Serilog и могут сохраняться в файловую директорию контейнера.

Рисунок 2.11 - Блок-схема процесса авторизации пользователя

3 Реализация веб-приложения

3.1 Реализация серверной части

Серверная часть реализована как ASP.NET Core Web API. Точка входа приложения выполняет настройку Serilog, регистрацию сервисов, инициализацию хранилища данных, подключение middleware-конвейера, операционных эндпоинтов и контроллеров. При запуске применяется миграция базы данных и проверяется наличие корневой директории загрузок.

Регистрация зависимостей вынесена в расширение

AddStoryDbApplication. В нем подключаются сервисы проектов, объектов, каталогов, атрибутов, структур, отношений, таймлайна, снимков проекта, экспорта, аутентификации, текущего пользователя, доступа к проектам, файлового хранилища, кэширования, аудита и метрик.

Для базы данных используется пул контекстов Entity Framework Core. В настройках PostgreSQL включено разбиение запросов, таймаут команд, повторные попытки подключения и параметры пула соединений. Это повышает устойчивость при работе в контейнерной среде.

Сервисный слой реализует предметные операции: создание проектов, создание и редактирование объектов, управление атрибутами, каталогами и

группами, построение графа отношений, сохранение компоновки графа, создание структур, назначение объектов на узлы, работа с событиями таймлайна и экспорт досье.

Валидация входных данных выполняется как на клиенте, так и на сервере. Серверная проверка необходима, так как клиентская валидация может быть обойдена. В проекте выделены RequestValidators и специализированные валидаторы, например валидатор событий таймлайна.

Конвейер обработки запросов включает middleware для контекста логирования, централизованной обработки исключений, ограничения размера тела запроса, сжатия ответов, CORS, проверки небезопасного происхождения запроса, аутентификации, rate limiting, защиты доступа к загруженным файлам, аудита и авторизации. Благодаря этому сквозные технические задачи не смешиваются с кодом контроллеров.

Сервис файлового хранилища инкапсулирует работу с директориями загрузок. API не должен знать физические детали размещения файлов, кроме публичного пути и метаданных. Такой подход упрощает дальнейшее изменение способа хранения без переписывания предметной логики.

Для фоновых операций используется hosted service. В частности, очередь экспорта позволяет принять запрос пользователя, вернуть идентификатор задания и выполнить формирование файла асинхронно. Это важно, потому что генерация DOCX может занимать больше времени, чем обычный запрос чтения.

Таблица 3.1 - Модули серверной части приложения

3.2 Реализация пользовательского интерфейса

Клиентская часть реализована на React и TypeScript. Приложение использует маршрутизацию, ленивую загрузку крупных модулей и разделение интерфейса на компоненты. Основной экран StoryDB содержит верхнюю панель, панель проекта, боковую навигацию и рабочую область выбранного раздела.

Модуль API Основная ответственность

AuthController Регистрация, вход, выход, получение и обновление профиля

ProjectsController Создание, редактирование, удаление и список проектов

ObjectsController Карточки объектов, галереи и краткие представления

CatalogsController Каталоги, записи, группы, поля и группы полей

StructuresController Структуры, узлы, использования и назначения объектов

RelationsController Граф связей и сохранение пользовательской раскладки

TimelineController События, связи событий, галереи, настройки и раскладка таймлайна

ProjectSnapshotsControlle

r Сборка, публикация и получение снимков проекта

ExportsController Синхронный и фоновый экспорт DOCX-досье

UploadsController Загрузка и проверка изображений

Рабочая область базы данных включает карточки объектов, редакторы, панели атрибутов, каталоги и структуры. Автор может создавать проект, добавлять персонажей, предметы, места и организации, заполнять основные поля, выбирать изображение, добавлять характеристики и привязывать объект к каталогам или структурам.

Для объектов предусмотрены как карточное, так и списочное представление. Карточное представление удобно для визуального просмотра художественного мира, списочное - для быстрого поиска и сравнения.

Вспомогательные функции формируют полные имена, инициалы, подписи связей и легкие представления объектов для больших списков.

Каталоги реализованы как настраиваемые справочники. Пользователь может создавать каталог, добавлять поля разных типов, объединять поля в группы, создавать группы записей и записи с пользовательскими значениями.

Это позволяет описывать разные предметные области: расы, классы, фракции, технологии, языки, артефакты, законы мира.

В интерфейсе учтена возможность работы с большими проектами.

Реализованы поиск, фильтрация активных объектов, виртуализация списков, устойчивые размеры карточек и разделение сложных форм на вкладки. Это снижает визуальную перегрузку и делает приложение пригодным для длительной ежедневной работы.

Маршрутизация клиента позволяет открывать конкретный раздел проекта по прямой ссылке. Например, можно перейти к базе персонажей, к странице отношений, к каталогу или к конкретному объекту. Это удобно для повторного открытия рабочего состояния и для передачи ссылок между участниками проекта.

Диалоговые окна используются для операций создания и редактирования, которые не требуют полного ухода с текущего экрана. Это помогает сохранить контекст автора: он видит список или граф, открывает форму редактирования, сохраняет изменения и возвращается к тому же рабочему месту.

Визуальные компоненты проекта разделены по областям: карточки объектов, детали объектов, каталоги, структуры, отношения, таймлайн, настройки, профиль и диалоги. Такое разделение снижает размер отдельных компонентов и позволяет развивать модули независимо.

3.3 Реализация графа связей, структур и временной линии

Одной из ключевых функций StoryDB является визуальная работа со связями. Граф отношений строится на основе объектов и связей между ними.

Узлами графа выступают сущности художественного мира, а ребрами - отношения владения, членства, территориального расположения, сюжетной связи, организационной принадлежности и другие зависимости.

Для графа предусмотрено сохранение пользовательской раскладки. Это важно, потому что автоматически построенный граф не всегда отражает авторскую логику. Пользователь может переместить узлы так, чтобы визуальная схема соответствовала его восприятию мира, а сервер сохраняет координаты и признак закрепления.

Структуры позволяют моделировать иерархии и графовые схемы. Они используются для организаций, территорий, социальных слоев, магических систем и других многосоставных сущностей. Один и тот же объект может участвовать в разных структурах, что важно для сложных сюжетных миров.

Временная линия реализует события разных типов: точечные события, длительные периоды, эпохи и главы. События могут иметь участников, изображения, категории, цвета, связи с другими событиями и изменения состояния объектов. Это позволяет отслеживать не только

последовательность событий, но и влияние событий на мир.

В клиенте реализованы функции разрешения временного состояния.

При выборе контекста события система может показывать объект с учетом изменений, произошедших к этому моменту: новый статус, принадлежность к каталогу, владение предметом, положение в структуре или отношения с другим персонажем. Такой механизм особенно важен для произведений с

длинной хронологией.

Для таймлайна предусмотрена генерация раскладки. Модель учитывает

типы событий, длительность, главы, горизонтальные и вертикальные связи,

независимые точки и правила размещения. Это позволяет визуально

разделять события и сохранять читаемость временной схемы.

При построении графа важно не смешивать разные категории связей в

одну неразличимую линию. Поэтому ребра имеют категорию, тип отношения

и дополнительные значения. Для персонажей могут использоваться сила и

напряженность отношения; для структур - роль в узле; для владения и

территорий - направление принадлежности.

Механизм временного состояния решает типичную авторскую

проблему: один и тот же объект может иметь разные характеристики в разных

главах. Например, персонаж в начале произведения является учеником, затем

изгнанником, затем лидером организации. Система позволяет хранить

базовую карточку и применять изменения из событий таймлайна для

выбранного момента.

События таймлайна связаны не только порядком, но и типами

отношений между событиями. Можно фиксировать причинную связь,

одновременность, предшествование, включение в более крупное событие и

свободную связанность. Это делает временную модель полезной не только

для календаря, но и для анализа логики сюжета.

3.4 Реализация медиафайлов, снимков проекта и экспорта

StoryDB поддерживает загрузку изображений для проектов, объектов,

записей каталогов и событий. На стороне клиента выполняется подготовка

изображений: крупные файлы могут быть уменьшены и нормализованы

перед отправкой, что снижает нагрузку на сервер и ускоряет работу

пользователя.

На сервере медиафайлы сохраняются через абстракцию файлового

хранилища. Для загруженных изображений фиксируются путь, тип

содержимого, хеш, размер, исходное имя и связь с проектом. Такой подход

упрощает дальнейшее расширение хранилища, например перенос файлов в

объектное хранилище.

Снимки проекта используются для фиксации согласованного состояния

данных. Текущий снимок предназначен для пользователя с правами доступа,

опубликованный снимок - для публичного чтения. Снимок хранит

агрегированные данные проекта в JSONB, что позволяет быстро отдавать

клиенту целостное представление без большого числа отдельных запросов.

Экспорт DOCX-досье реализован как отдельный сервис. Пользователь

выбирает объекты и параметры включения данных: атрибуты, каталоги,

отношения, структуры. Сервер формирует документ в формате DOCX с

помощью OpenXML SDK. Для тяжелых операций предусмотрен фоновый

экспорт через очередь заданий и скачивание результата по завершении.

Наличие экспорта особенно важно для практического применения.

Автор может подготовить набор досье для редактора, соавтора, художника,

сценариста или участника команды, не предоставляя полный доступ к

рабочей базе данных.

Экспортируемое досье должно быть читаемым без доступа к веб-

приложению. Поэтому в документ включаются основные сведения об

объекте, атрибуты, каталожные связи, отношения и структурные назначения.

Такой документ может использоваться как техническое задание для

иллюстратора, справка для редактора или рабочая выжимка для автора перед

написанием сцены.

Снимки проекта и экспорт решают разные задачи. Снимок фиксирует состояние данных для быстрого чтения и публикации внутри веб-системы.

Экспорт создает автономный документ, который можно отправить внешнему участнику процесса. Совместное наличие этих механизмов повышает практическую применимость приложения.

3.5 Развертывание приложения

Для развертывания используется Docker Compose. Конфигурация включает три основных сервиса: PostgreSQL, серверный API и клиентское приложение. База данных запускается на образе postgres:17-alpine, API собирается из Dockerfile серверного проекта, клиент собирается из Dockerfile клиентского проекта и обслуживается через Nginx.

В docker-compose.yml определены переменные окружения для подключения к базе данных, настройки памяти, лимитов загрузок, параметров очередей экспорта, лимитов кэша, путей загрузок, логов и ключей защиты данных. Это позволяет адаптировать систему под разные условия запуска без изменения исходного кода.

Для базы данных используется volume, сохраняющий состояние PostgreSQL между перезапусками контейнеров. Для загруженных файлов, ключей Data Protection и логов используются монтируемые директории. Такой подход отделяет изменяемые данные от образов приложения. Готовность базы данных проверяется healthcheck-ом PostgreSQL, а API запускается после перехода базы в состояние healthy. На стороне API реализованы собственные эндпоинты live, ready и health, которые могут использоваться для мониторинга и диагностики.

4 Тестирование и оценка результатов

4.1 Методика тестирования

Проверка приложения выполняется на нескольких уровнях. Unit-тесты проверяют отдельные сервисы, валидаторы, вспомогательные функции и алгоритмы. Интеграционные тесты проверяют работу API, взаимодействие с базой данных, аутентификацию, создание проектов, работу с объектами, каталогами, структурами, экспортом, загрузками и снимками проекта.

Для клиентской части используются тесты доменных функций: построение дерева групп каталога, подготовка изображений, маршрутизация, фильтрация объектов, расчет виртуальных списков, разрешение временного состояния и валидация форм. E2E smoke-тесты проверяют, что пользовательский интерфейс открывается, защищенные маршруты корректно обрабатывают анонимного пользователя, а клиент может обращаться к API.

В репозитории подготовлено 30 тестовых файлов и 127 тестовых сценариев по маркерам Fact, Theory, test и it. Наличие тестов не заменяет ручную приемку интерфейса, но позволяет автоматически контролировать критичные функции и предотвращать регрессии при изменении кода.

Unit-тестирование особенно полезно для функций, которые имеют четко определенные входы и выходы: валидация, расчет виртуального окна списка, построение дерева групп, форматирование отображаемых имен, подготовка изображений и разрешение временного состояния. Такие функции можно проверять без запуска всего приложения.

Интеграционные тесты необходимы для проверки полного цикла запроса. Они позволяют убедиться, что контроллер, сервис, база данных, аутентификация и правила доступа работают согласованно. Для StoryDB это важно, потому что многие операции завязаны на идентификатор проекта и

права текущего пользователя.

E2E smoke-тестирование выполняет роль раннего индикатора проблем клиентской сборки и маршрутизации. Если приложение открывается пустым экраном или защищенные маршруты неправильно обрабатывают анонимный доступ, такие проверки должны обнаружить проблему до ручной приемки.

Таблица 4.1 - Уровни проверки приложения

4.2 Проверка функциональных сценариев

Для оценки готовности приложения выделены основные пользовательские сценарии. Они отражают путь автора от создания проекта до подготовки экспортируемых материалов. Проверка таких сценариев позволяет убедиться, что отдельные модули работают не изолированно, а в составе единого приложения.

Таблица 4.2 - Проверка функциональных сценариев

Уровень проверки Примеры проверяемых компонентов

Unit-тесты API Валидаторы, кэш single-flight, метрики, сервисы связей и структур

Интеграционные тесты

API

Аутентификация, проекты, объекты, каталоги, экспорт,

снимки, загрузки, health endpoints

Тесты клиента Фильтрация, маршруты, временное состояние,

подготовка изображений, дерево групп каталога

E2E smoke Открытие приложения, доступность API проху,

защищенные маршруты, отсутствие пустого экрана

Нагрузочные smoke-

проверки Сценарии k6 и скрипты локальной проверки готовности

Сценарий Ожидаемый результат

Регистрация и вход Пользователь создает учетную запись, входит в систему и получает сессию

Создание проекта Создается проект с выбранными типами объектов и режимом видимости

Заполнение карточки

персонажа

Сохраняются основные поля, статус, атрибуты,

изображение и связи

Создание каталога Пользователь создает справочник, поля, группы и записи каталога

Построение структуры Создается структура с узлами, ребрами и назначением объектов

Работа с графом связей Отображаются узлы и ребра, сохраняется

пользовательская раскладка

Создание события

таймлайна

Событие сохраняется, связывается с участниками и

влияет на временное состояние

Публикация снимка Формируется согласованное состояние проекта для

4.3 Оценка соответствия требованиям

Реализованное приложение соответствует основным функциональным требованиям. Пользователь может создавать проекты, вести карточки объектов, настраивать атрибуты и каталоги, описывать отношения, формировать структуры, работать с временной линией, загружать

изображения, публиковать снимки и экспортировать досье.

Нефункциональные требования также учтены. Архитектура разделена

на клиентскую и серверную части, код организован по предметным модулям,

база данных развивается через миграции, предусмотрены проверки входных

данных, ограничения запросов, аутентификация, авторизация, аудит, метрики,

контейнерное развертывание и фоновые задания.

По сравнению с хранением материалов в обычных документах или

таблицах StoryDB дает автору более связную модель проекта. Данные о

персонажах, предметах, местах, организациях, каталогах и событиях

находятся в одной системе. Это снижает вероятность расхождения сведений и

ускоряет поиск нужной информации.

Особенно значимым результатом является объединение статической и

динамической моделей. Система хранит карточки объектов, но также

позволяет анализировать граф отношений и изменения во времени. Такой

подход лучше соответствует природе художественного проекта, где важны не

только описания, но и развитие событий.

Результат работы можно оценивать не только по наличию отдельных

функций, но и по степени связности данных. Объект, созданный в системе,

может одновременно иметь текстовое описание, изображение,

характеристики, каталоговые признаки, отношения, участие в событиях,

текущего или публичного просмотра

Экспорт досье Формируется DOCX-файл по выбранным объектам и

параметрам включения данных

Проверка эксплуатации Эндпоинты live, ready, health и metrics возвращают

диагностическую информацию

структурные назначения и отражение в экспортируемом досье. Это

показывает, что приложение формирует единую информационную среду

проекта.

К ограничениям текущей версии можно отнести отсутствие

полноценного многопользовательского редактирования в реальном времени,

ограниченные средства импорта из внешних систем и отсутствие встроенного

редактора текста произведения. Эти ограничения не отменяют достижения

цели работы, так как основной задачей являлась разработка базы знаний и

инструментов моделирования, а не полноценной среды написания текста.

Таблица 4.3 - Оценка соответствия функциональным требованиям

4.4 Перспективы развития

Дальнейшее развитие StoryDB может включать совместное

редактирование в реальном времени, расширенную систему ролей, историю

Требование Статус

выполнения Комментарий

Единая база объектов Выполнено

Поддержаны персонажи, предметы,

места, организации и расширяемые

типы

Настраиваемые

характеристики Выполнено Реализованы атрибуты, группы и типы

значений

Каталогизация Выполнено Каталоги имеют поля, группы, записи и

иерархические режимы

Граф отношений Выполнено Есть получение графа и сохранение

раскладки

Таймлайн Выполнено Поддержаны события, связи событий, участники и изменения

Медиафайлы Выполнено Есть загрузка изображений и хранение вариантов

Экспорт DOCX Выполнено Реализован синхронный и фоновый экспорт досье

Безопасность Выполнено Сессии, доступ к проектам, лимиты и аудит

Развертывание Выполнено Docker Compose объединяет БД, API и клиент

Совместная работа в реальном времени Перспектива Может быть добавлена в следующих версиях

изменений с возможностью отката, комментарии редактора, импорт данных из популярных форматов заметок и таблиц, а также расширенные средства анализа сюжетных противоречий.

Перспективным направлением является развитие текстового модуля. В текущем виде приложение ориентировано на базу знаний и моделирование мира. В дальнейшем можно добавить редактор сцен, связь сцен с объектами и событиями, автоматическую проверку упоминаний персонажей и экспорт материалов в форматы, удобные для редакторов и издательств.

Также возможно развитие визуальной части: улучшение автоматической компоновки графов, поддержка нескольких представлений одного графа, фильтрация связей по категориям и времени, печатные схемы, а также гибкая настройка вида таймлайна для разных жанров.

Отдельное направление развития связано с публикацией. Система уже поддерживает публичные снимки проекта, но в дальнейшем можно создать режим публичной энциклопедии произведения, где часть материалов доступна читателям, а рабочие сведения остаются закрытыми для автора.

Заключение

В ходе выполнения выпускной квалификационной работы была рассмотрена проблема информационной поддержки авторов художественных произведений. Установлено, что при разработке крупного сюжетного проекта автору необходимо хранить и анализировать множество взаимосвязанных данных: персонажей, предметы, места, организации, каталоги, события, отношения и временные изменения.

Был выполнен анализ существующих подходов к ведению авторских материалов. Показано, что текстовые документы, таблицы и универсальные заметочки удобны для отдельных задач, но не обеспечивают единой предметной модели художественного мира. Специализированные сервисы ближе к задаче, однако не всегда дают необходимую гибкость, контроль над данными и возможность локального развертывания.

На основе анализа были сформулированы требования к веб-приложению StoryDB. Система должна поддерживать проекты, карточки объектов, настраиваемые атрибуты, каталоги, структуры, граф отношений, временную линию, загрузку изображений, экспорт досье, безопасность и эксплуатационную диагностику.

Была спроектирована и реализована клиент-серверная архитектура. Серверная часть создана на ASP.NET Core с использованием Entity Framework Core и PostgreSQL. Клиентская часть реализована на React и TypeScript. Для развертывания подготовлена Docker Compose-конфигурация,

включающая базу данных, API и клиентское приложение.

В практической части реализованы основные модули: управление пользователями и проектами, база объектов, атрибуты, каталоги, структуры, граф отношений, таймлайн, загрузка медиафайлов, снимки проекта, аудит, метрики и экспорт DOCX-досье. Реализация содержит развитую модель данных, набор контроллеров API, сервисный слой и компоненты пользовательского интерфейса.

Для проверки результата подготовлены unit, integration, client-domain и E2E smoke-тесты. Они покрывают валидацию, работу API, аутентификацию, проекты, объекты, каталоги, структуры, экспорт, снимки, загрузки, временное состояние и основные маршруты клиентского приложения.

Таким образом, цель работы достигнута: разработано веб-приложение для авторов художественных произведений, позволяющее системно хранить материалы проекта, анализировать связи и хронологию, уменьшать риск противоречий и подготавливать структурированные материалы для дальнейшей работы.

Список использованных источников

1. Microsoft. ASP.NET Core documentation. URL: <https://learn.microsoft.com/aspnet/core/> (дата обращения: 22.06.2026).
2. Microsoft. Entity Framework Core documentation. URL: <https://learn.microsoft.com/ef/core/> (дата обращения: 22.06.2026).
3. PostgreSQL Global Development Group. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата обращения: 22.06.2026).
4. React. React Documentation. URL: <https://react.dev/> (дата обращения: 22.06.2026).
5. TypeScript. TypeScript Documentation. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 22.06.2026).
6. Vite. Vite Guide. URL: <https://vite.dev/guide/> (дата обращения: 22.06.2026).
7. Docker. Docker Compose documentation. URL: <https://docs.docker.com/compose/> (дата обращения: 22.06.2026).
8. Open XML SDK documentation. URL: <https://learn.microsoft.com/office/open-xml/open-xml-sdk> (дата обращения: 22.06.2026).
9. SixLabors. ImageSharp documentation. URL: <https://docs.sixlabors.com/> (дата обращения: 22.06.2026).
10. OWASP Foundation. OWASP Application Security Verification Standard. URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 22.06.2026).
11. Фаулер М. Архитектура корпоративных программных приложений. М.: Вильямс, 2019.
12. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. СПб.: Питер, 2020.
13. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. СПб.: Питер, 2021.
14. Мартин Р. Чистый код. Создание, анализ и рефакторинг. СПб.: Питер, 2020.
15. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. СПб.: Питер, 2021.
16. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.
17. ГОСТ 19.701-90. Единая система программной документации. Схемы

алгоритмов, программ, данных и систем.

18. Исходный код проекта StoryDB: серверная часть, клиентская часть, тесты и конфигурация развертывания. Локальный репозиторий проекта.

19. Документация проекта StoryDB: шаблоны экспорта, статьи, эксплуатационные материалы и скрипты развертывания. Локальный репозиторий проекта.

Приложение А

Перечень иллюстраций для последующего дополнения

В основной текст работы добавлены проектные схемы и схемы базы данных. После подготовки экранных форм приложения рекомендуется дополнить работу скриншотами пользовательского интерфейса и обновить нумерацию страниц.

Таблица А.1 - Рекомендуемые иллюстрации для последующего дополнения

Раздел Рекомендуемая иллюстрация

2.4 Главная рабочая область StoryDB с проектом и боковой навигацией

3.2 Форма редактирования объекта художественного мира

3.2 Рабочая область каталогов и пользовательских полей

3.3 Граф отношений между персонажами и объектами

3.3 Временная линия событий проекта

3.4 Окно экспорта DOCX-досье

3.5 Схема контейнерного развертывания Docker Compose

4.2 Пример результата функциональной проверки пользовательского сценария